LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi Antar Kelas

NAMA: Ken Rayana Ilma Madani

NIM: 103132430008

KELAS: S1 SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Diagram Kelas

   Diagram kelas merupakan sebuah diagram yang digunakan untuk memodelkan kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut.

2. Hubungan Antar Kelas

   Dalam Object Oriented Programming, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

   a. Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, harus ada hubungan
   b. Agregasi merupakan hubungan antara dua kelas yang lebih ketat dan terdapat hubungan bagian (part) dan keseluruhan (whole).
   c. Komposisi merupakan bentuk khusus dari agregasi dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi part (bagian) tidak akan ada tanpa adanya kelas yang menjadi whole.

B. Guided

1. Guided01 Asosisi

```
//Dosen.Java
        package guided.guided1;

        public class Dosen {
            private String kodeDosen;
            private String namaDosen;

            //Setter
            public void setKodeDosen(String kodeDosen) {
                this.kodeDosen = kodeDosen;
            }

            public void setNamaDosen(String namaDosen) {
                this.namaDosen = namaDosen;
            }

            //Getter
            public String getKodeDosen() {
                return this.kodeDosen;
            }

            public String getNamaDosen() {
                return this.namaDosen;
            }
```

```
    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}

//Mahasiswa.Java
    package guided.guided1;

    public class Mahasiswa {
        private String nim;
        private String nama;
        private int nilai;

        //Setter
        public void setNama(String nama) {
            this.nama = nama;
        }

        public void setNim(String nim) {
            this.nim = nim;
        }

        public String getNama() {
            return this.nama;
        }

        public void setNilai(int nilai){
            this.nilai = nilai;
        }

        //Getter
        public int getNilai(){
            return this.nilai;
```

```
        }


        public String getNim() {
            return this.nim;
        }
}
```

Output :

```
--- compiler:3.15.0:compile (default-compile) @ ProjectModul5 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul5 ---
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.921 s
Finished at: 2026-10-19T10:55:15+07:00
------------------------------------------------------------------------
```

2. Guided 2 Agregasi

```
//Jurusan.java
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");
```

```
        for (int i=0; i<jumlahMahasiswa;i++){
            System.out.println(i+1 + "," +
            daftarMahasiswa[i].getNIM() + " "
            + daftarMahasiswa[i].gerNama());
        }
    }

    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }
    //Setter

    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
//Mahasiswa.Java
package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }
    public String gerNama(){
        return Nama;
    }

    //setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
```

```
    }
}
//ProjectModul5.java(main)
package com.kenrayana.projectmodul5;

import guided.agregasi.*;
import guided.komposisi.*;

public class ProjectModul5 {

    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103132430008", "Ken Rayana");
        Mahasiswa m2 = new Mahasiswa("103132430010", "John Dae");

        Jurusan prodi = new Jurusan("SD", "SainsData");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch", "Logitech",
"Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul5 ---
Kode Jurusan: SD
Nama Jurusan: SainsData
Daftar Mahasiswa:
1,103132430008 Ken Rayana
2,103132430010 John Dae
```

3. Guided 3 Komposisi

```
//Komputer.Java
        /*
         * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
        default.txt to change this license
         * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
        edit this template
```

```
         */
        package guided.komposisi;

        public class Komputer {
            private Processor processor;
            private Monitor monitor;
            private Mouse mouse;

            public Komputer(String brandCpu, int kecepatanCpu, String
        brandMonitor, String brandMouse, String tipeMouse) {
                this.processor = new Processor(brandCpu, kecepatanCpu);
                this.monitor = new Monitor(brandMonitor);
                this.mouse = new Mouse(brandMouse, tipeMouse);
            }

            public void displaySpecKomputer() {
                System.out.println("=== SPESIFIKASI KOMPUTER ===");
                this.processor.displaySpec();
                this.monitor.DisplaySpec();
                this.mouse.displaySpec();
            }
}
//Monitor.Java

        package guided.komposisi;

        public class Monitor {
            private String brand;

            public Monitor(String brand) {
                this.brand = brand;
            }

            public void DisplaySpec() {
                System.out.println(this.brand);
            }
}
//Mouse.Java
        package guided.komposisi;

        public class Mouse {
            private String brand, tipe;

            public Mouse(String brand, String tipe) {
                this.brand = brand;
                this.tipe = tipe;
```

```
        }

        public void displaySpec() {
            System.out.println(this.brand + ", " + this.tipe);
        }
}
//Processor.Java
    package guided.komposisi;

    public class Processor {
        private String brand;
        private int kecepatan;

        //constructor
        public Processor(String brand,int kecepatan){
            this.brand = brand;
            this.kecepatan = kecepatan;
        }

        //behaviour
        public void displaySpec(){
            System.out.println(this.brand + ", " + this.kecepatan);
        }
}
//ProjectModul5.Java (Main)
    package guided.agregasi;

    public class Jurusan {
        private static int jumlahMahasiswa;
        private String kodeJurusan, namaJurusan;
        private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

        //constructor
        public Jurusan (String kodeJurusan, String namaJurusan){
            this.kodeJurusan = kodeJurusan;
            this.namaJurusan = namaJurusan;
        }

        //Behaviour
        public void addMahasiswa(Mahasiswa m){
            this.daftarMahasiswa[jumlahMahasiswa] = m;
            jumlahMahasiswa++;
        }

        public void displayMahasiswa(){
            System.out.println("Kode Jurusan: " + this.kodeJurusan);
```

```
            System.out.println("Nama Jurusan: " + this.namaJurusan);

            System.out.println("Daftar Mahasiswa: ");

            for (int i=0; i<jumlahMahasiswa;i++){
                System.out.println(i+1 + "," +
                daftarMahasiswa[i].getNIM() + " "
                + daftarMahasiswa[i].gerNama());
            }
        }

        //Getter
        public String getKodeJurusan(){
            return kodeJurusan;
        }

        public String getNamaJurusan(){
            return namaJurusan;
        }
        //Setter

        public void setKodeJurusan(String kodeJurusan){
            this.kodeJurusan = kodeJurusan;
        }

        public void setNamaJurusan(String namaJurusan){
            this.namaJurusan = namaJurusan;
        }
}
```

Output:

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

   Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

   Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

   Jawaban :

```
//ItemPesanan.Java
package com.kenrayana.projectmodul5.unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // KOMPOSISI: RincianHarga dibuat di dalam constructor ItemPesanan
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}

 // Constructor 3: lengkap (memakai setter agar validasi tetap berlaku)
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int jumlahMissing) {
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    // Method milik class: bisa dipanggil tanpa membuat objek
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter
    public String getNama() {
        return nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public int getJumlahMissing() {
        return jumlahMissing;
```

```
    }

    // Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    // Nilai negatif ditolak -> nilai lama tidak berubah
    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    // Persentase sel kosong terhadap total sel (baris x kolom)
    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0;
        }
        return (double) jumlahMissing / totalSel * 100;
    }

    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}

//Kasir.Java
package com.kenrayana.projectmodul5.unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // ASOSIASI: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai field
```

```
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] daftar = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = daftar[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Pesanan.Java

```
package com.kenrayana.projectmodul5.unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // AGREGASI: menampung ItemPesanan dari luar
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak dapat
ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
```

```
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
//RincianHarga.Java
package com.kenrayana.projectmodul5.unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 10 / 100; // pajak 10% dari harga dasar
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
//ProjectModul5.Java (Main)
package com.kenrayana.projectmodul5;

import guided.agregasi.*;
import guided.komposisi.*;
import com.kenrayana.projectmodul5.unguided.*;
```

```
public class ProjectModul5 {

    public static void main(String[] args) {
        runGuided2();
        runGuided3();
        runUnguided();
    }

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103132430008", "Ken Rayana");
        Mahasiswa m2 = new Mahasiswa("103132430010", "John Dae");

        Jurusan prodi = new Jurusan("SD", "SainsData");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch", "Logitech",
"Wireless");
        pc.displaySpecKomputer();
    }

    public static void runUnguided() {
        // 1. Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. ItemPesanan dibuat independen (RincianHarga dibuat otomatis di dalamnya)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Pesanan Meja 4, kapasitas 5 item
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Masukkan item ke pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan pesanan
        pesanan.tampilkanPesanan();

        // 6. Proses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.461 s
Finished at: 2026-10-19T11:19:40+07:00
------------------------------------------------------------------------
```

Penjelasan :

Program ini mensimulasikan pencatatan dan pembayaran pesanan di sebuah cafe, dengan tujuan melatih tiga jenis relasi antar kelas: komposisi, agregasi, dan asosiasi. Ketiganya dibedakan oleh seberapa kuat keterikatan antar objek dan siapa yang membuat objeknya. Program terdiri dari lima class. RincianHarga menyimpan harga dasar dan pajak 10% serta menghitung harga total. ItemPesanan mewakili satu menu yang dipesan, lengkap dengan rincian harganya. Pesanan menjadi wadah yang menampung beberapa item untuk satu meja. Kasir memproses pembayaran dan mencetak struk. Main (di project gabungan ada di method runUnguided()) merangkai semuanya dan menguji program.

Relasi pertama adalah komposisi, yaitu antara ItemPesanan dan RincianHarga. Rincian harga tidak masuk akal berdiri sendiri karena harga selalu milik suatu item, sehingga objek RincianHarga dibuat dengan new di dalam constructor ItemPesanan, bukan di main. Jika ItemPesanan dihapus, RincianHarga-nya ikut hilang.

Relasi kedua adalah agregasi, yaitu antara Pesanan dan ItemPesanan. Setiap ItemPesanan dibuat lebih dulu secara terpisah di main, lalu dimasukkan ke Pesanan lewat method tambahItem() dan disimpan di array daftarItem. Karena dibuat dari luar, item tetap ada secara konsep walaupun pesanannya dibatalkan.

Relasi ketiga adalah asosiasi, yaitu antara Kasir dan Pesanan. Kasir tidak menyimpan Pesanan sebagai field dan hanya punya namaKasir. Objek Pesanan masuk sebagai parameter pada prosesPembayaran(Pesanan pesanan), sehingga hubungan keduanya hanya ada selama method itu berjalan.

Saat dijalankan, alurnya begini. Pertama dibuat Kasir bernama Budi, lalu tiga ItemPesanan (Iced Americano, Beef Croissant, Matcha Latte) yang masing-masing otomatis membuat RincianHarga dengan pajak 10%. Setelah itu dibuat Pesanan untuk Meja 4 dengan kapasitas array 5, dan ketiga item dimasukkan lewat tambahItem(). Method ini mengecek kapasitas dulu, mengisi array, lalu menaikkan jumlahItem. Berikutnya tampilkanPesanan() menampilkan nomor meja dan tiap item beserta harga total, harga dasar, dan pajaknya. Terakhir, prosesPembayaran(pesanan) menjumlahkan seluruh harga dasar dan pajak lalu mencetak struk. Perhitungannya: subtotal 18.000 + 25.000 + 22.000 = 65.000, pajak 10% = 6.500, sehingga total bayar 71.500.

Beberapa konsep pendukung juga dipakai. Enkapsulasi diterapkan dengan menjadikan semua field private dan mengaksesnya lewat getter, sehingga Kasir mengambil data pesanan lewat getRincian() dan getJumlahItem(), bukan langsung ke field. Keyword this dipakai karena nama parameter constructor sama dengan nama field, misalnya this.namaItem = namaItem. Array standar ItemPesanan[] berukuran tetap dipakai sebagai pengganti ArrayList, sehingga jumlahItem dibutuhkan untuk menandai berapa slot yang sudah terisi. Terakhir, pajak dihitung dengan hargaDasar * 10 / 100, bukan * 0.1, supaya tidak terkena error pembulatan tipe double.

D. Kesimpulan

Modul 5 ini membahas hubungan antar kelas dalam pemrograman berorientasi objek, yaitu asosiasi, agregasi, dan komposisi. Dari praktikum ini dapat disimpulkan bahwa ketiganya berbeda pada tingkat keterikatan antar objek dan pada siapa yang bertanggung jawab membuat objeknya. Pada asosiasi, dua objek saling menggunakan lewat parameter method tanpa saling memiliki. Pada agregasi, sebuah objek menampung objek lain yang dibuat secara independen di luar sehingga objek tersebut tetap bisa berdiri sendiri. Pada komposisi, objek bagian dibuat di dalam constructor objek utamanya dan tidak bisa ada tanpanya.